U.S. General Services Administration

# Agentic AI Book Club - Week 10

## Reviewing Chapter 2.5 & 2.6

AI Community of Practice - 2026

# Agenda

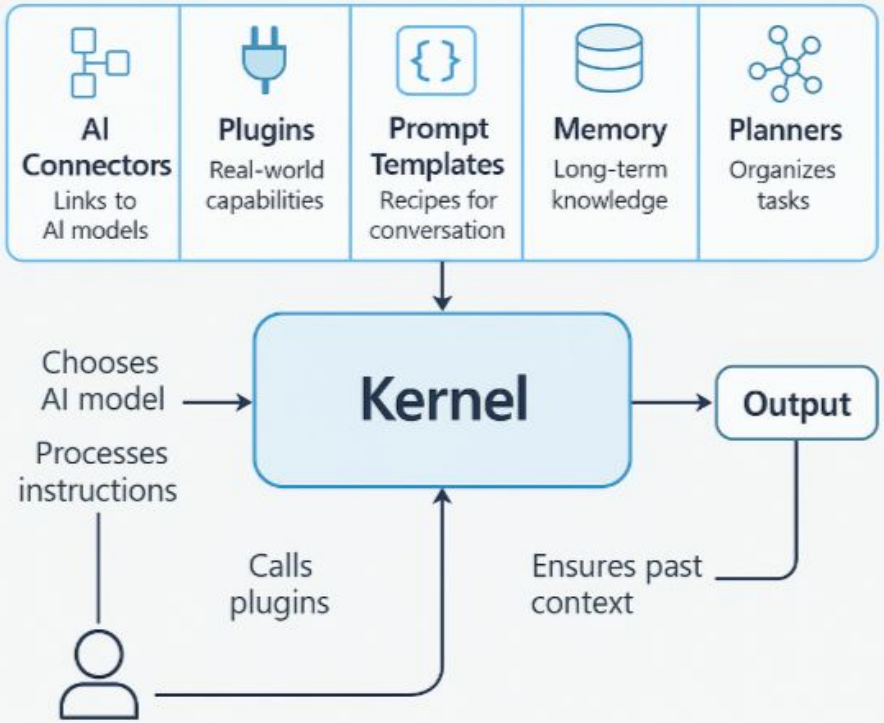
- Housekeeping
- Chapter review
- ~~Quiz review~~
- End of Section 1 Test - DEMO
- Intro to next week’s chapters

# Housekeeping

- **Agentic AI is an early and rapidly evolving knowledge domain**
- End of Section 1 Test will be in Google form and will be available from 01 to 15 June
- CPEs will be eligible for Cohort 1 Section 2 and Cohort 2 Section 1 → more time to prepare → Scheduled for early July.
- Look out for communications from us mid June

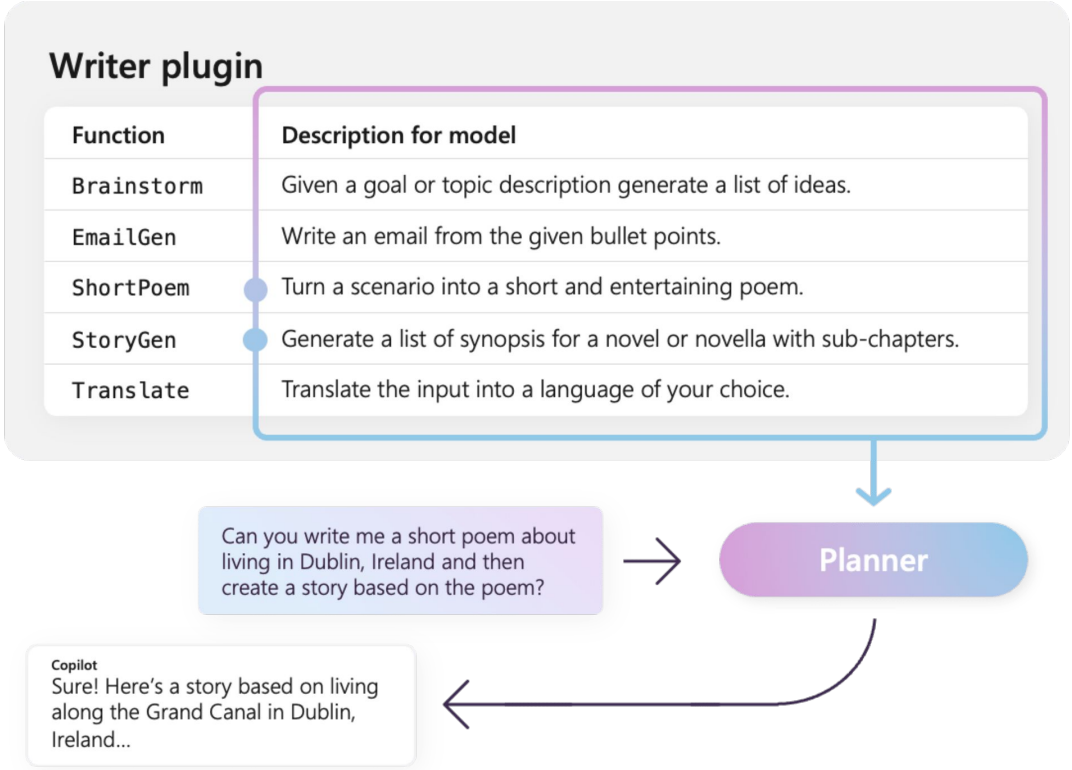
# Chapter Review

# The Kernel -- Central Orchestration Hub



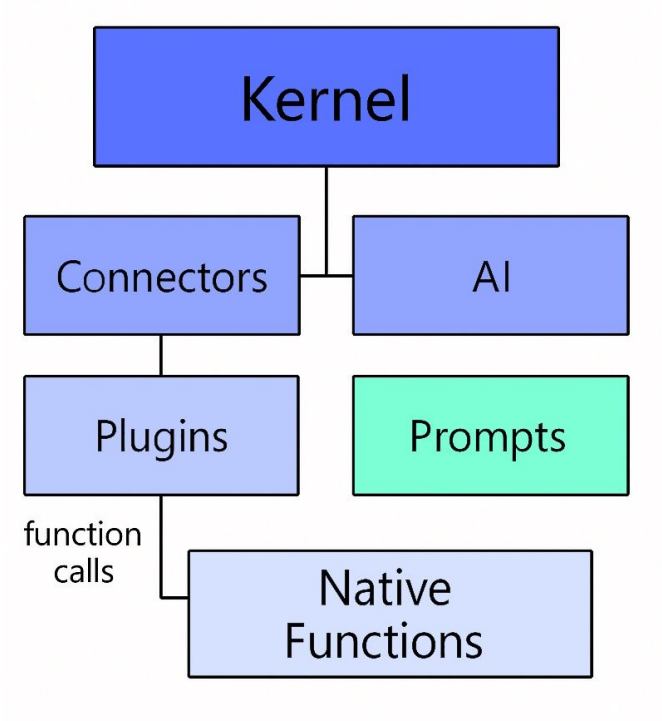

Read more: https://medium.com/microsoftazure/step-by-step-guide-to-develop-ai-multi-agent-system-using-microsoft-semantic-kernel-and-gpt-4o-f5991af40ea6

# Plugin Architecture -- Modular Capabilities



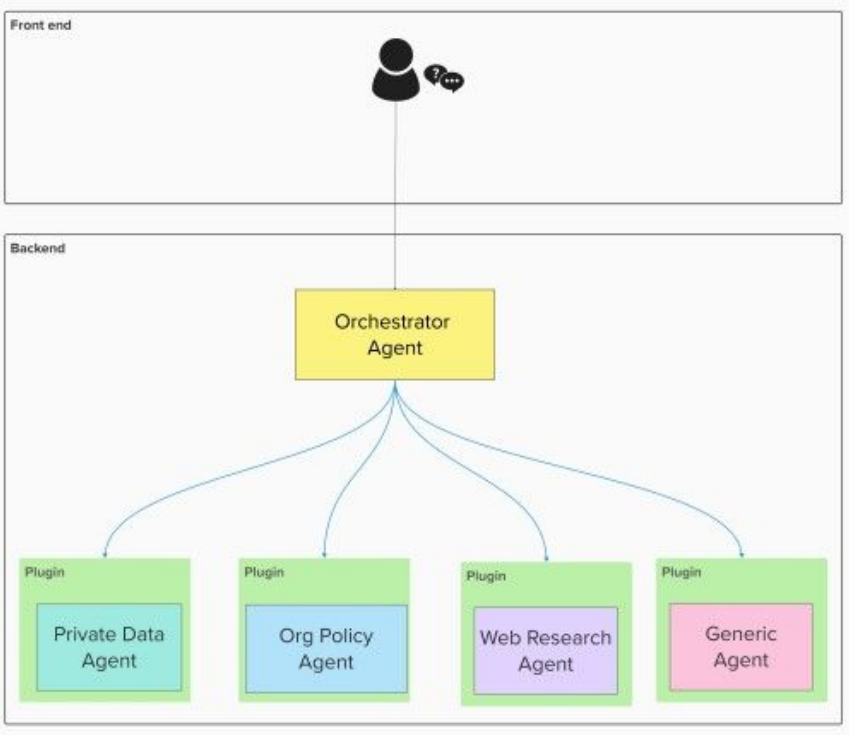

Credit: https://learn.microsoft.com/en-us/semantic-kernel/concepts/plugins/?pivots=programming-language-csharp

# Semantic vs. Native Functions




- Native functions: deterministic code for data and APIs
- Semantic functions: LLM prompts for reasoning and generation
- Native retrieves accurate data from systems of record
- Semantic analyzes data and generates human responses
- Combined: intelligent responses with data accuracy

# LLM-Driven Dynamic Routing




- Kernel builds system prompt describing all available functions
- LLM analyzes queries and decides which plugins to invoke
- No hard-coded workflow sequences required
- Multi-step workflows emerge from LLM reasoning
- Adapts routing when initial results need more info

Credit: https://devblogs.microsoft.com/agent-framework/guest-blog-orchestrating-ai-agents-with-semantic-kernel-plugins-a-technical-deep-dive/

# Controlling the Flexibility-Determinism Tradeoff

- Dynamic routing is non-deterministic even at temperature zero
- FunctionChoiceBehavior configures routing constraints
- Auto mode: maximum flexibility, accepts non-determinism
- Required mode: forces specific functions for compliance
- Plugin filtering narrows routing to relevant domains

# When to Choose Semantic Kernel

- Extensive capability catalogs needing dynamic composition
- Microsoft Azure ecosystem provides native integration
- Plugin reusability across multiple enterprise agents
- Centralized observability for regulated industries
- Teams familiar with dependency injection patterns

# When to Avoid Semantic Kernel

- Single-purpose agents with fewer than 5 capabilities
- Workflows requiring deterministic execution paths
- Teams with Python data science backgrounds, not enterprise
- LangChain handles simple agents with less boilerplate
- LangGraph better suited for strict control flow needs

# Function Calling Mechanics

- LLM generates structured JSON recommending a function call
- Application code parses, validates, and executes the call
- LLM never executes functions directly -- only recommends
- Result returns to LLM as context for the final response
- Works across OpenAI, Anthropic, and Google with minor syntax differences

# The Critical Misconceptions

Misconceptions

- LLMs can hallucinate function names and parameters
- Temperature=0 does not guarantee deterministic outputs

Why?

- Tokenization and floating-point variance cause subtle drift
- Treat all LLM function call output as untrusted input
- Validate everything before execution

# Tool Schema Design

- JSON Schema is the industry standard for tool specifications
- Input schemas define types, constraints, required fields
- Output schemas tell the LLM what data to expect back
- Works across LangChain, Semantic Kernel, and cloud platforms
- OpenAPI specs enable automatic tool generation from APIs

# Tool Description Engineering

- Description quality directly impacts selection accuracy
- Specify when to use, what it does, and what it returns
- Include input format examples and edge cases
- Vague descriptions cause unpredictable tool selection
- Think of descriptions as contracts, not comments

# Tool Chaining -- Sequential Dependencies

- Chain tools when Step N+1 needs output from Step N
- LLM acts as orchestrator, routing data between tools
- State managed through conversation history or state objects
- Validate success at each step before proceeding
- Failures cascade silently through the chain

# State Management Across Tool Chains

- AgentExecutor maintains history and intermediate results
- LangGraph makes state flow explicit through graph nodes
- Context windows overflow as chains accumulate state
- Format transformations needed between incompatible tools
- Shared memory layers enable multi-agent tool workflows

# NVIDIA NIM for Tool-Heavy Agents

- NIM accelerates inference up to 5x with TensorRT-LLM
- Speculative decoding predicts likely function names
- Batch processing shares GPU across multiple requests
- GPU utilization jumps from 20-40% to 70-90%
- API-compatible with OpenAI and Anthropic tool formats

# Production Checklist for Tool Integration

- Write detailed tool descriptions with usage examples
- Validate all LLM-generated function calls before execution
- Implement error handling that returns messages, not exceptions
- Log every tool call for debugging and auditing
- Test with multiple providers if targeting cross-platform deployment

# Next Chapters

- Chapter 2.7: Multimodal RAG - Integration of Vision, Audio, and Text

  The chapter covers vision model specialization, image routing logic, Whisper-based audio transcription with time indexing, and the NVIDIA multimodal stack for production deployment.
- Chapter 2.8: Error Handling and Resilience

  The chapter addresses framework integration with LangChain and LangGraph, provides worked examples of resilient multi-tool agents, and explains how to achieve 99.9% uptime through comprehensive pattern application.
- Chapter 2.9: Streaming and Real-Time Responses

  The chapter covers the perceived latency principle where sub-second feedback matters more than total latency, Time to First Token optimization, protocol selection between Server-Sent Events and WebSockets, and LangServe integration for production streaming infrastructure.

# End of Section Test

## Ranking Perception Demo

# Review Materials

- Ch 1.4: Perception Systems, Multimodal Input Processing, Temporal Synchronization
- Ch 1.7A: Entity Recognition, Relationship Extraction from unstructured text
- Ch 2.7: Multimodal RAG architectures (visual, audio, text fusion)
- Ch 1.1B: Perceivability — accessible design for diverse input needs

# Correctly Define “Perception”

Perception encompasses how an agent senses, acquires, and preprocesses input from its environment. This includes the types of input modalities processed (text, images, audio, documents, sensor data), the quality and complexity of signals, real-time processing requirements, and the sophistication of the pipeline that converts raw inputs into structured, actionable representations.

# Craft Key Evaluation Dimensions

1. Input modality count and diversity (text, images, audio, video, structured data, sensor streams)
2. Signal quality and preprocessing complexity (clean vs. noisy, adversarial inputs)
3. Real-time vs. batch processing requirements
4. Multi-modal fusion and temporal alignment needs
5. Adversarial robustness and edge case handling

# Define Complexity Levels

- **Level 1** (simple): Single structured text input; no real-time requirement; human-curated, clean data | Single text field input (user query), pre-cleaned structured datasets, static document processing, known/fixed schemas

- **Level 3** (moderately complex): Multiple modalities requiring alignment, or single modality with adversarial/noisy conditions, or real-time stream processing | Document + table + figure extraction, news article surveillance from heterogeneous web sources, real-time text stream monitoring, NLP entity extraction pipelines

- **Level 5** (extremely complex): Adversarial multi-modal inputs in real-time, safety-critical perception, no human oversight, high failure cost | Autonomous vehicle perception, battlefield situational awareness, medical imaging + vitals + EHR fusion under time pressure, adversarial document injection defense |

# Grade and Rank

The test will ask you relative rankings among use cases, not the exact levels for each use case.